

Goals

The learning objective of this lab is to become familiar with Osquery and its detection capabilities.

What you will learn

You will learn how to hunt for linux process injection leveraging Osquery.

Recommended tools

- Osquery
- <https://github.com/kolide/fleet>

NOTE - Before you can begin the lab,

1. Execute **start.sh** script to enroll the endpoint to Kolide Fleet.

```
cd /root/Desktop/osquery-client/  
./start.sh
```

```

root@kali:~# cd /root/Desktop/osquery-client/
root@kali:~/Desktop/osquery-client# ./start.sh
Starting osquery-client_centos7-osquery_1 ... done
Starting osquery-client_ubuntu16-osquery_1 ... done
Starting osquery-client_ubuntu20-osquery_1 ... done
Starting osquery-client_centos8-osquery_1 ... done
Starting osquery-client_ubuntu14-osquery_1 ... done
Recreating target-machine ... done
Starting osquery-client_centos6-osquery_1 ... done
Attaching to target-machine, osquery-client_ubuntu20-osquery_1, osquery-client_centos7-osquery_1, osquery-client_ubuntu16-osquery_1, osquery-client_centos6-osquery_1, osquery-client_centos8-osquery_1
target-machine | I0630 04:50:58.163115 | 1 init.cpp:418] osquery initialized [version=4.1.2]
target-machine | I0630 04:50:58.163362 | 1 system.cpp:362] Writing osqueryd pid (1) to /var/run/osqueryd
target-machine | I0630 04:50:58.164263 | 1 extensions.cpp:349] Could not autoload extensions: Failed read
target-machine | I0630 04:50:58.165289 | 7 watcher.cpp:583] osqueryd watcher (1) executing worker (8)
target-machine | I0630 04:50:58.176018 | 8 init.cpp:415] osquery worker initialized [watcher=1]
target-machine | I0630 04:50:58.177389 | 8 rocksdb.cpp:131] Opening RocksDB handle: /var/osquery/osquery
target-machine | I0630 04:50:58.219978 | 8 tls_enroll.cpp:67] TLSEnrollPlugin requesting a node enroll
i/v1/osquery/enroll
target-machine | I0630 04:50:58.220613 | 15 interface.cpp:268] Extension manager service starting: /var/c
target-machine | I0630 04:50:58.395916 | 8 smbios_tables.cpp:104] Reading SMBIOS from sysfs DMI node
target-machine | I0630 04:50:58.397625 | 8 smbios_tables.cpp:104] Reading SMBIOS from sysfs DMI node
target-machine | I0630 04:50:58.397917 | 8 tls.cpp:253] TLS/HTTPS POST request to URI: https://10.0.5.10
target-machine | I0630 04:50:58.446556 | 8 auto_constructed_tables.cpp:93] Removing stale ATC entries
target-machine | I0630 04:50:58.447943 | 8 tls.cpp:253] TLS/HTTPS POST request to URI: https://10.0.5.10
ubuntu14-osquery_1 | I0630 04:50:58.472295 | 1 init.cpp:418] osquery initialized [version=4.1.2]
target-machine | I0630 04:50:58.495021 | 8 smbios_tables.cpp:104] Reading SMBIOS from sysfs DMI node
ubuntu20-osquery_1 | I0630 04:50:58.495828 | 1 init.cpp:418] osquery initialized [version=4.1.2]
target-machine | I0630 04:50:58.496747 | 8 smbios_tables.cpp:104] Reading SMBIOS from sysfs DMI node
target-machine | I0630 04:50:58.548497 | 8 events.cpp:863] Event publisher not enabled: auditeventpublis
uration

```

You should now be able to see the endpoint inside Kolide fleet (name: target-machine)

Note: While accessing the Kolide fleet ignore the certificate warning.

URL <https://10.0.5.10:8412/login>

Username: admin

Password: P@\$w0rd123321

Kolide Fleet

← → ↻ 🏠

🔒 https://10.0.5.10:8412/hosts/manage

 **INE**
ADMIN

HOSTS

QUERY

PACKS

HELP

ADMIN

All Hosts

DESCRIPTION

All hosts which have enrolled in Fleet

7 Hosts Total

7bfac1d14377

Ubuntu 18.4.0

4.1.2

2 x 2.3 GHz

7.8 GB

11 minutes

02:42:AC:12:00:06

172.18.0.6

7c56538a17ff

Ubuntu 14.4.0

4.1.2

2 x 2.3 GHz

7.8 GB

6 minutes

02:42:AC:12:00:06

172.18.0.6

95d718066408

CentOS Linux 8.1.1911

4.1.2

2 x 2.3 GHz

7.8 GB

6 minutes

02:42:AC:12:00:04

172.18.0.4

969d684ff9a9

Ubuntu 20.4.0

4.1.2

2 x 2.3 GHz

7.8 GB

6 minutes

02:42:AC:12:00:07

172.18.0.7

f5e6f03c0946

Ubuntu 16.4.0

4.1.2

2 x 2.3 GHz

7.8 GB

6 minutes

02:42:AC:12:00:02

172.18.0.2

target-machine

Ubuntu 18.4.0

4.1.2

2 x 2.3 GHz

7.8 GB

6 minutes

02:42:AC:12:00:08

172.18.0.8

Shell No. 1

Kolide Fleet — Mozi...

Hunting At Scale With Osquery

LAB 18

Scenario

In this lab you will learn how to hunt for process injection on Linux endpoints with the help of Osquery. You will also be shown how to execute pre-baked Osquery queries through Kolide fleet to hunt for process injection at scale.

If you are unfamiliar with Osquery. Please refer to our Medium post and video training below.

<https://medium.com/@d.bougioukas/blue-team-diary-entry-1-leveraging-osquery-for-enhanced-incident-response-threat-hunting-70935538c9c3>

Goals

The learning objective of this lab is to become familiar with Osquery and its detection capabilities.

What you will learn

You will learn how to hunt for linux process injection leveraging Osquery.

Recommended tools

- Osquery
- <https://github.com/kolide/fleet>

NOTE - Before you can begin the lab,

1. Execute **start.sh** script to enroll the endpoint to Kolide Fleet.

```
cd /root/Desktop/osquery-client/  
./start.sh
```

```

root@kali:~# cd /root/Desktop/osquery-client/
root@kali:~/Desktop/osquery-client# ./start.sh
Starting osquery-client_centos7-osquery_1 ... done
Starting osquery-client_ubuntu16-osquery_1 ... done
Starting osquery-client_ubuntu20-osquery_1 ... done
Starting osquery-client_centos8-osquery_1 ... done
Starting osquery-client_ubuntu14-osquery_1 ... done
Recreating target-machine ... done
Starting osquery-client_centos6-osquery_1 ... done
Attaching to target-machine, osquery-client_ubuntu20-osquery_1, osquery-client_centos7-osquery_1, osquery-client_ubuntu16-osquery_1, osquery-client_centos6-osquery_1, osquery-client_centos8-osquery_1
target-machine | I0630 04:50:58.163115 | 1 init.cpp:418] osquery initialized [version=4.1.2]
target-machine | I0630 04:50:58.163362 | 1 system.cpp:362] Writing osqueryd pid (1) to /var/run/osqueryd
target-machine | I0630 04:50:58.164263 | 1 extensions.cpp:349] Could not autoload extensions: Failed read
target-machine | I0630 04:50:58.165289 | 7 watcher.cpp:583] osqueryd watcher (1) executing worker (8)
target-machine | I0630 04:50:58.176018 | 8 init.cpp:415] osquery worker initialized [watcher=1]
target-machine | I0630 04:50:58.177389 | 8 rocksdb.cpp:131] Opening RocksDB handle: /var/osquery/osquery
target-machine | I0630 04:50:58.219978 | 8 tls_enroll.cpp:67] TLSEnrollPlugin requesting a node enroll
i/v1/osquery/enroll
target-machine | I0630 04:50:58.220613 | 15 interface.cpp:268] Extension manager service starting: /var/
target-machine | I0630 04:50:58.395916 | 8 smbios_tables.cpp:104] Reading SMBIOS from sysfs DMI node
target-machine | I0630 04:50:58.397625 | 8 smbios_tables.cpp:104] Reading SMBIOS from sysfs DMI node
target-machine | I0630 04:50:58.397917 | 8 tls.cpp:253] TLS/HTTPS POST request to URI: https://10.0.5.10
target-machine | I0630 04:50:58.446556 | 8 auto_constructed_tables.cpp:93] Removing stale ATC entries
target-machine | I0630 04:50:58.447943 | 8 tls.cpp:253] TLS/HTTPS POST request to URI: https://10.0.5.10
ubuntu14-osquery_1 | I0630 04:50:58.472295 | 1 init.cpp:418] osquery initialized [version=4.1.2]
target-machine | I0630 04:50:58.495021 | 8 smbios_tables.cpp:104] Reading SMBIOS from sysfs DMI node
ubuntu20-osquery_1 | I0630 04:50:58.495828 | 1 init.cpp:418] osquery initialized [version=4.1.2]
target-machine | I0630 04:50:58.496747 | 8 smbios_tables.cpp:104] Reading SMBIOS from sysfs DMI node
target-machine | I0630 04:50:58.548497 | 8 events.cpp:863] Event publisher not enabled: auditeventpublis
uration

```

You should now be able to see the endpoint inside Kolide fleet (name: target-machine)

Note: While accessing the Kolide fleet ignore the certificate warning.

URL <https://10.0.5.10:8412/login>

Username: admin

Password: P@\$w0rd123321

Kolide Fleet

← → ↻ 🏠

🔒 https://10.0.5.10:8412/hosts/manage

 **INE**

 **ADMIN**

HOSTS

QUERY

PACKS

HELP

ADMIN

All Hosts

DESCRIPTION

All hosts which have enrolled in Fleet

7 Hosts Total

7bfac1d14377

Ubuntu 18.4.0

4.1.2

2 x 2.3 GHz

7.8 GB

11 minutes

02:42:AC:12:00:06

172.18.0.6

7c56538a17ff

Ubuntu 14.4.0

4.1.2

2 x 2.3 GHz

7.8 GB

6 minutes

02:42:AC:12:00:06

172.18.0.6

95d718066408

CentOS Linux 8.1.1911

4.1.2

2 x 2.3 GHz

7.8 GB

6 minutes

02:42:AC:12:00:04

172.18.0.4

969d684ff9a9

Ubuntu 20.4.0

4.1.2

2 x 2.3 GHz

7.8 GB

6 minutes

02:42:AC:12:00:07

172.18.0.7

99ce15cc5d66

CentOS Linux 7.7.1908

4.1.2

2 x 2.3 GHz

7.8 GB

6 minutes

02:42:AC:12:00:03

172.18.0.3

f5e6f03c0946

Ubuntu 16.4.0

4.1.2

2 x 2.3 GHz

7.8 GB

6 minutes

02:42:AC:12:00:02

172.18.0.2

target-machine

Ubuntu 18.4.0

4.1.2

2 x 2.3 GHz

7.8 GB

6 minutes

02:42:AC:12:00:08

172.18.0.8

🖱️ 📁 🔍 🚪

Shell No. 1

Kolide Fleet — Mozi...

Tasks

Task 1. Emulate and hunt for library injection using Osquery

Access the **target-machine** Ubuntu container and use the sample target and injection tool residing at */root/Tools/linux-inject* to emulate library injection against a Linux process.

Then, try to identify artifacts/traces that can be used to hunt for library injection against a Linux process.

Identify the Osquery tables or functionality that contain information regarding the artifacts/traces you came up with.

Do the same with the injection tool that resides at */root/Tools/ReflectiveSOInjection*

Hints:

- https://osquery.io/schema/3.3.2#system_controls
- https://osquery.io/schema/3.3.2#process_memory_map

Task 2. Hunt for library injection using at scale using Kolide Fleet

Perform or schedule the Osquery queries you came up with during Task 1 from inside Kolide Fleet. This way you can perform the same query over multiple endpoints at once...

SOLUTIONS

Below, you can find solutions for each task. Remember though, that you can follow your own strategy, which may be different from the one explained in the following lab.

Task 1. Emulate and hunt for library injection using Osquery

First, let's access the container and start our *sample-target* process.

```
docker exec -it target-machine bash
cd /root/Tools/linux-inject
./sample-target
```

```
root@kali:~# docker exec -it target-machine bash
root@target-machine:~# cd /root/Tools/linux-inject
root@target-machine:~/Tools/linux-inject# ./sample-target
sleeping...
sleeping...
sleeping...
sleeping...
█
```

Leave this session running and [open a new container session](#) and perform the injection as follows.

```
docker exec -it target-machine bash
cd /root/Tools/linux-inject
./inject -n sample-target sample-library.so
```

```
root@kali:~# docker exec -it target-machine bash
root@target-machine:~# cd /root/Tools/linux-inject
root@target-machine:~/Tools/linux-inject# ./inject -n sample-target sample-library.so
targeting process "sample-target" with pid 52
"sample-library.so" successfully injected
root@target-machine:~/Tools/linux-inject# █
```

The injection performed successfully.

This is what you will see inside the SSH session where *sample-target* is running.

```
sleeping...
sleeping...
sleeping...
sleeping...
sleeping...
sleeping...
sleeping...
I just got loaded
sleeping...
sleeping...
sleeping...
sleeping...
```

Let's continue with the detection part, as follows.

```
osqueryi
select * from system_controls WHERE name == 'kernel.yama.ptrace_scope';
```

```
root@target-machine:~/Tools/linux-inject# osqueryi
Using a virtual database. Need help, type '.help'
osquery> select * from system_controls WHERE name == 'kernel.yama.pttrace_scope';
+-----+-----+-----+-----+-----+-----+
| name | oid | subsystem | current_value | config_value | type |
+-----+-----+-----+-----+-----+-----+
| kernel.yama.pttrace_scope |  | kernel | 0 | 1 | string |
+-----+-----+-----+-----+-----+-----+
osquery> █
```

Still inside the osquery "shell", execute the below.

```
SELECT process_memory_map.*, pid as mpid from process_memory_map LEFT JOIN processes USING (pid) WHERE process_memory_map.path LIKE '/' and process_memory_map.pseudo != 1 AND process_memory_map.path NOT LIKE '/lib/%' AND process_memory_map.path NOT LIKE '/usr/lib%' AND process_memory_map.path != processes.path AND process_memory_map.permissions LIKE '%x%';
osquery> SELECT process_memory_map.*, pid as mpid from process_memory_map LEFT JOIN processes USING (pid) WHERE process_memory_map.path LIKE '/' and process_memory_map.pseudo != 1 AND process_memory_map.path NOT LIKE '/lib/%' AND process_memory_map.path NOT LIKE '/usr/lib%' AND process_memory_map.path != processes.path AND process_memory_map.permissions LIKE '%x%';
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| pid | start | end | permissions | offset | device | inode | path | pseudo |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 52 | 0x7f3bfc5cf000 | 0x7f3bfc5d0000 | r-xp | 0 | ca:01 | 1082770 | /root/Tools/linux-inject/sample-library.so | 0 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
osquery> █
```

The above query looks at the process memory using Osquery's *process_memory_map* table. We are interested in memory locations that have the execute permission. The original image as well as locations marked as pseudo should be disregarded. We should also remove known paths where libraries are loaded. Now, delete the injected library (you can exit osqueryi by executing *.exit*).

```
CTRL + C
ls
rm sample-library.so
```

```
osquery> ^C

root@target-machine:~/Tools/linux-inject# ls
LICENSE.txt  README.md  inject-arm.c  inject-x86_64.c  ptrace.c  sample-library.c  sample-library32.so  sample-target.c  slides_BHArse...
Makefile    inject    inject-x86.c  inject32        ptrace.h  sample-library.so  sample-target        sample-target32  utils.c
root@target-machine:~/Tools/linux-inject# rm sample-library.so
root@target-machine:~/Tools/linux-inject# █
```

Execute *osqueryi* again and issue the query below.

```
osqueryi
SELECT process_memory_map.pid, process_memory_map.start, process_memory_map.end, process_memory_map.permissions, process_memory_map.offset, process_memory_map.path from process_memory_map LEFT
join file USING (path) where pseudo != 1 AND process_memory_map.path NOT LIKE '/lib/%' AND process_memory_map.path NOT LIKE '/usr/lib%' AND process_memory_map.permissions LIKE '%x%' AND
filename IS NULL and process_memory_map.inode !=0 AND process_memory_map.permissions = 'r-xp';
```

```
root@target-machine:~/Tools/linux-inject# osqueryi
Using a virtual database. Need help, type '.help'
osquery> SELECT process_memory_map.pid, process_memory_map.start, process_memory_map.end, process_memory_map.permissions, process_memory_map.path from process_memory_map LEFT join file USING (path) where pseudo != 1 AND process_memory_map.path NOT LIKE '/usr/lib%' AND process_memory_map.permissions LIKE '%x%' AND filename IS NULL and process_memory_map.inode < 1000000;
+-----+-----+-----+-----+-----+-----+
| pid | start          | end          | permissions | offset | path                                     |
+-----+-----+-----+-----+-----+-----+
| 52  | 0x7f3bfc5cf000 | 0x7f3bfc5d0000 | r-xp        | 0      | /root/Tools/linux-inject/sample-library.so |
+-----+-----+-----+-----+-----+-----+
osquery> █
```

The injected library

is still visible! Terminate osqueryi. Let's try the other injection tool. Terminate *sample-target* and start it again inside the other SSH session.

```
cd /root/Tools/ReflectiveSOInjection/inject/src
ps aux (to identify the PID of sample-target)
./inject -p PID_of_sample_target ../../so/src/ReflectiveLoader.so
```

```

root@target-machine:~/Tools/linux-inject# cd /root/Tools/ReflectiveSOInjection/inject/src
root@target-machine:~/Tools/ReflectiveSOInjection/inject/src# ps aux
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.0  0.2 126920 18096 ?        Ssl  04:50   0:00 osqueryd --flagfile=/etc/osquery/osquery.flags --tls_hostname=10.0.5
root         8  0.1  0.3 1180616 27824 ?        SNl   04:50   0:01 osqueryd
root        42  0.0  0.0  18512   3388 pts/0    Ss   04:57   0:00 bash
root       52  0.0  0.0   6568   1500 pts/0    S+   04:59   0:00 ./sample-target
root       53  0.0  0.0  18512   3420 pts/1    Ss   04:59   0:00 bash
root       75  0.0  0.0  34404   2796 pts/1    R+   05:06   0:00 ps aux
root@target-machine:~/Tools/ReflectiveSOInjection/inject/src# ./inject -p 52 ../../so/src/ReflectiveLoader.so
[i] targeting process with pid 52
[+] shared object mapped at 0x7f49ad298000
[+] found dynamic segment at 0x7f49ad29be20
[+] dynsym found at address 0x7f49ad298250
[+] dynstr found at address 0x7f49ad298418
[+] Resolved ReflectiveLoader offset to 0x7aa
[i] Setting target registers to appropriate values
[i] Overwriting target memory region with shellcode
[+] Transferring execution to stage 0 shellcode
[+] Returned from Stage 0 shell code RIP of target is 0x40006f
[i] Stage 0 mmap returned memory address of 0x7f3bfcddde000.. verifying allocation succeeded..
[+] Okay.. mmap allocation was successful!
[+] Writing our shared object into the victim process address space MUAHAHAHA!!!
[+] Setting RIP to ReflectiveLoader function
[+] Calling ReflectiveLoader function! Let's hope this works ;D
root@target-machine:~/Tools/ReflectiveSOInjection/inject/src#

```

Let's detect the library injection, again using osquery.

```

osqueryi
SELECT processes.name, process_memory_map.*, pid as mpid from process_memory_map join processes USING (pid) WHERE process_memory_map.permissions = 'rwxp' AND process_memory_map.path = '';

```

```

root@target-machine:~/Tools/ReflectiveSOInjection/inject/src# osqueryi
Using a virtual database. Need help, type '.help'
osquery> SELECT processes.name, process_memory_map.*, pid as mpid from process_memory_map join processes USING (pid) WHERE
rwxp' AND process_memory_map.path = '';
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| name          | pid | start          | end          | permissions | offset | device | inode | path | pseudo | mpid |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| sample-target | 52  | 0x7f3bfbfc9000 | 0x7f3bfc3cb000 | rwxp        | 0      | 00:00  | 0     |      | 0      | 52   |
| sample-target | 52  | 0x7f3bfcdde000 | 0x7f3bfcde5000 | rwxp        | 0      | 00:00  | 0     |      | 0      | 52   |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
osquery> █

```

advantage of the second library injection technique is that the injected shared object doesn't have to be on disk. That's why the query above searches for memory regions marked as rwxp having an empty path.

The

Task 2. Hunt for library injection using at scale using Kolide Fleet

First, log into Kolide Fleet and create a new query, as follows.

The screenshot shows the 'New Query' form in the INE Admin interface. The 'Query Title' field is filled with 'ptrace_scope_tampering'. The 'SQL' field contains the query: `select * from system_controls WHERE name == 'kernel.yama.ptrace_scope';`. The 'Description' field is empty. The 'Select Targets' dropdown is set to 'Label Name, Host Name, IP Address, etc.'. The 'SAVE' button is highlighted with a red box.

Inside SQL, enter the below query (the one we used before to detect the first library injection tool).

```
select * from system_controls WHERE name == 'kernel.yama.ptrace_scope';
```

Select Targets

Label Name, Host Name, IP Address, etc.

ALL HOSTS

 All Hosts

7 hosts



LABELS

 macOS

0 hosts



 Ubuntu Linux

5 hosts



 CentOS Linux

2 hosts



 MS Windows

0 hosts



HOSTS

 7c56538a17ff • 172.18.0.6



 f5e6f03c0946 • 172.18.0.2



99ce15cc5d66 • 172.18.0.3



95d718066408 • 172.18.0.4



 969d684ff9a9 • 172.18.0.7



Then, select all Ubuntu hosts, by pressing the + sign next to them and finally click SAVE (as new) press the database icon on your left.

This is what you should see if you

 **INE**

 **ADMIN**



 HOSTS

 QUERY

• **Manage Queries**

New Query

 PACKS

 HELP

 ADMIN

1 Queries

 Filter Queries

CREATE NEW QUERY

☐	Query Name	Author	Last Modified
☐	ptrace_scope_tampering	admin	06/30/22

Now click the folder icon and then click "create new pack".

 **INE**
ADMIN

HOSTS

QUERY

PACKS

• Manage Packs

New Pack

?

HELP

ADMIN

o Query Packs

Search Packs

CREATE NEW PACK

☐	Pack Name	Queries	Status	Hosts	Last Modified
No packs available. Try creating one.					

 **INE**
ADMIN

HOSTS

QUERY

PACKS

Manage Packs

- New Pack

HELP

ADMIN

New Pack

Query Pack Title

ptrace_scope_tampering

Query Pack Description

Add a description of your query

Select Pack Targets

Label Name, Host Name, IP Address, etc.

ALL HOSTS

 All Hosts 7 hosts



LABELS

 macOS 0 hosts



 Ubuntu Linux 5 hosts



 CentOS Linux 2 hosts



 MS Windows 0 hosts



Once again add all Ubuntu hosts and click Save.
clicking the folder icon.

This is what you should see when

 **INE**
ADMIN

HOSTS

QUERY

PACKS

• Manage Packs

New Pack

?

HELP

ADMIN

1 Query Packs

Search Packs

CREATE NEW PACK

☐	Pack Name	Queries	Status	Hosts	Last Modified
☐	ptrace_scope_tampering	0	 Enabled	5	06/30/22

Click on the pack you created and then click "Edit Pack" on your right.

FileEditViewHistoryBookmarksToolsHelp

Kolide Fleet

←→↻🏠https://10.0.5.10:8412/packs/manage?selectedPack=1

K

INE

ADMIN

HOSTS

QUERY

PACKS

Manage Packs

New Pack

HELP

ADMIN

1

Query Packs

Search Packs

1 Pack Selected

✖️ DISABLE

✅ ENABLE

🗑️ DELETE

	Pack Name	Queries	Status	Hosts	Last Modified
☒	ptrace_scope_tampering	0	✅ Enabled	5	06/30/22

KOLIDE

root@target-machi...Kolide Fleet — Mozi...

Choose the ptrace-related query you created from the menu on your right and configure as follows.

Kolide Fleet

← → ↻ 🏠

K

INE

ADMIN

HOSTS

QUERY

PACKS

Manage Packs

New Pack

?

HELP

ADMIN

pttrace_scope_tampering

EDIT

select pack targets

5 unique hosts

Ubuntu Linux

0 Queries

Search Queries

☐	Query Name	Interval [s]	Platform	⚙ Ver.	Shard	Logging
First let's add a query. Then we'll set the following: interval: the amount of time, in seconds, the query waits before running platform: the computer platform where this query will run (other platforms ignored) minimum ⚙ version: the minimum required osqueryd version installed on a host logging type: +/- differential: show only what's added from last run + differential (ignore removals): show only what's been added since the last run 📷 snapshot: show everything in its current state						

root@target-machi...

Kolide Fleet — Mozi...

Kolide Fleet

← → ↻ 🏠

K

INE

ADMIN

HOSTS

QUERY

PACKS

Manage Packs

New Pack

?

HELP

ADMIN

pttrace_scope_tampering

EDIT

select pack targets

5 unique hosts

Ubuntu Linux

0 Queries

Search Queries

☐	Query Name	Interval [s]	Platform	⚙ Ver.	Shard	Logging
First let's add a query. Then we'll set the following: interval: the amount of time, in seconds, the query waits before running platform: the computer platform where this query will run (other platforms ignored) minimum ⚙ version: the minimum required osqueryd version installed on a host logging type: +/- differential: show only what's added from last run + differential (ignore removals): show only what's been added since the last run 📷 snapshot: show everything in its current state						

root@target-machi...

Kolide Fleet — Mozi...

Finally, click Save at the bottom.

Kolide Fleet

← → ↻ 🏠

K

INE

ADMIN

HOSTS

QUERY

PACKS

Manage Packs

New Pack

HELP

ADMIN

pttrace_scope_tampering

EDIT

select pack targets

5 unique hosts

Ubuntu Linux

1 Query

Search Queries

☐	Query Name	Interval [s]	Platform	Ver.	Shard	Logging
☐	pttrace_scope_tampering	60		Any		+/-

root@target-machi...

Kolide Fleet — Mozi...

Fleet logs are being pushed to an ELK instance at <http://10.0.5.10:5601> There you should be able to identify changes to the ptrace scope, at scale. For example, change back the ptrace scope on the *osquery_client* machine as follows and you should see a new event inside Kibana.

```
echo 1 | tee /proc/sys/kernel/yama/ptrace_scope
```

```
root@target-machine:~/Tools/ReflectiveSOInjection/inject/src# echo 1 | tee /proc/sys/kernel/yama/ptrace_scope
1
root@target-machine:~/Tools/ReflectiveSOInjection/inject/src#
```

[Go to Discover](#)

Kibana

10.0.5.10:5601/app/kibana#/home?_g=()

Home

Recently viewed

Discover

Visualize

Dashboard

Canvas

Maps

Machine Learning

Metrics

Logs

APM

Uptime

SIEM

Dev Tools

Stack Monitoring

Management

Add Data to Kibana

Use these solutions to quickly turn your data into pre-built dashboards and monitor your applications.

APM

APM automatically collects in-depth performance metrics and errors from inside your applications.

[Add APM](#)

Logging

Ingest logs from popular data sources and easily visualize in preconfigured dashboards.

[Add log data](#)

Add sample data

Load a data set and a Kibana dashboard

Upload data

Import a CSV

[Create an index](#)

Create index pattern

Kibana uses index patterns to retrieve data from Elasticsearch indices for things like visualizations.

☐ Include system indices

Step 1 of 2: Define index pattern

Index pattern

You can use a * as a wildcard in your index pattern.
You can't use spaces or the characters \, /, ?, ", <, >, |.

> Next step

✓ **Success!** Your index pattern matches **1 index**.

osquery-result

Rows per page: 10 ▾

In configure settings select @timestamp and Click on

Create index pattern

Kibana uses index patterns to retrieve data from Elasticsearch indices for things like visualizations.

☐ Include system indices

Step 2 of 2: Configure settings

You've defined **osquery-result** as your index pattern. Now you can specify some settings before we create it.

Time Filter field name Refresh

▼

@timestamp

—

I don't want to use the Time Filter

< Back

Create index pattern

create index pattern

Again go to Discover, you should

see the new events inside Kibana.

FileEditViewHistoryBookmarksToolsHelp

Discover - Kibana

10.0.5.10:5601/app/kibana#/discover?_g=()&_a=(columns:!(_source),index:b7632a40-f834-11ec-9cf2-ad6298f219f3,interval:auto,query:(language:kuery,query:''),sort:!(('@timestamp'>))

Discover

NewSaveOpenShareInspect

Search

KQL

Last 15 minutes

+ Add filter

osquery-result

Search field names

Filter by type0

Selected fields

</> _source

Available fields

@timestamp

@version

_id

_index

_score

_type

action

calendarTime

columns.config_value

columns.current_value

columns.name

columns.oid

12 hits

Jun 30, 2022 @ 10:38:35.370 - Jun 30, 2022 @ 10:53:35.370 — Auto

Count

@timestamp per 30 seconds

Time

_source

> Jun 30, 2022 @ 10:47:52.000

decorations.host_uuid: ec2dd514-e9d8-85ae-eb70-36bf828c8214 decorations.hostname: target-machine hostIdentifier: target-machine host: e0b618
name: pack/ptrace_scope_tampering/ptrace_scope_tampering @timestamp: Jun 30, 2022 @ 10:47:52.000 columns.config_value: 1 columns.current_val
columns.name: kernel.yama.ptrace_scope columns.oid: columns.subsystem: kernel columns.type: string type: osquery_json numerics: false uni
calendarTime: Thu Jun 30 05:17:52 2022 UTC @version: 1 path: /var/log/osquery/osqueryd.results.log epoch: 0 counter: 1 action: removed _id:
_index: osquery-result _score: -

> Jun 30, 2022 @ 10:47:52.000

decorations.host_uuid: ec2dd514-e9d8-85ae-eb70-36bf828c8214 decorations.hostname: target-machine hostIdentifier: target-machine host: e0b618
name: pack/ptrace_scope_tampering/ptrace_scope_tampering @timestamp: Jun 30, 2022 @ 10:47:52.000 columns.config_value: 1 columns.current_val
columns.name: kernel.yama.ptrace_scope columns.oid: columns.subsystem: kernel columns.type: string type: osquery_json numerics: false uni
calendarTime: Thu Jun 30 05:17:52 2022 UTC @version: 1 path: /var/log/osquery/osqueryd.results.log epoch: 0 counter: 1 action: added _id:
_index: osquery-result _score: -

> Jun 30, 2022 @ 10:47:27.000

decorations.host_uuid: ec2f4bd9-384e-d2cc-e5d9-482e6e234251 decorations.hostname: 969d684ff9a9 hostIdentifier: 969d684ff9a9 host: e0b6189c07

root@target-machi...

Discover - Kibana ...

Resources

1. <https://www.trustedsec.com/blog/linux-hows-my-memory/>
2. <https://cybersecurity.att.com/blogs/labs-research/hunting-for-linux-library-injection-with-osquery>

KEY TAKEAWAYS

1. There are numerous ways for an adversary to achieve their goal. Being familiar with as many as possible attacks can aid hunters in creating better detection rules performing successful hunts.
2. Hunting playgrounds are a great source of different attacks, helping hunters in increasing their abilities.